

Implementing Iterator and Iterable in Java

Building a custom, for-each-compatible data structure

1. Why Implement Iterable?

Any class that implements the `Iterable` interface correctly can be used directly in an enhanced for loop (for-each loop):

```
for (Item item : customDataStructure) {  
    // do stuff  
}
```

The need to implement `Iterator` specifically arises when designing **custom data structures** — built-in collections already come with iteration support, but a hand-rolled structure (a custom linked list, tree, graph, ring buffer, etc.) only gains for-each support once it explicitly implements `Iterable` and supplies an `Iterator`.

2. How to Implement an Iterable Data Structure

Making a data structure iterable takes two pieces working together:

- Implement the **Iterable** interface (and its `iterator()` method) on the data structure class itself.
- Create a separate **Iterator** class that implements the `Iterator` interface and its corresponding methods (`hasNext()`, `next()`, and optionally `remove()`).

The general shape of this pairing, independent of the underlying data structure, looks like this:

```
class CustomDataStructure implements Iterable<T> {  
    public Iterator<T> iterator() {  
        return new CustomIterator<>(this);  
    }  
}  
  
class CustomIterator<T> implements Iterator<T> {  
    CustomIterator(CustomDataStructure obj) { /* init cursor */ }  
    public boolean hasNext() { ... } // next element exists?  
    public T next() { ... } // advance cursor, return element  
    public void remove() { ... } // optional; throws by default
```

Note: the `Iterator` class can also be written as an inner class of the data structure class, since it is rarely, if ever, useful outside that context.

3. How next() and hasNext() Work

Implementing an `Iterator` requires a **cursor** (or pointer) that keeps track of which element is currently being pointed at. How the cursor advances from one element to the next depends entirely on the underlying data structure — an array-backed structure advances an integer index, while a linked structure follows a node reference.

`next()` is responsible for two things: returning the current element, and advancing the cursor to the following one. Crucially, `next()` should not be called blindly — `hasNext()` is always checked first to confirm an element actually exists. The behind-the-scenes pattern that a for-each loop expands into is essentially:

```
while (iterator.hasNext()) {    // if next element exists
    next();                    // advance the pointer
}
```

4. Initializing the Cursor

Cursor initialization depends on the data structure being iterated: in a linked list, the cursor starts at the **head** node; in an array-backed list, it starts at index **0**. How the cursor gets access to that starting point depends on how the Iterator class itself is structured:

- If the Iterator is an **inner class** of the data structure, it can reach the structure's fields directly using the qualified **this** keyword (e.g. `CustomDataStructure.this.element`).
- If the Iterator is a **separate, standalone class**, the data structure instance must be passed into the iterator's constructor so it has something to read from.

5. Worked Example: A Custom Generic Linked List

As an illustration, consider a custom singly linked list built from Generic **Node** objects, where each node holds a data value and a reference to the next node. The list class exposes **add()**, plus **getHead()/getTail()** accessors, and implements **Iterable** so it can supply its own iterator. A separate **ListIterator** class implements the **Iterator** interface to actually walk the nodes.

5.1 The list side

The list class implements **Iterable** and its **iterator()** method simply hands back a new instance of the accompanying iterator, seeded with a reference to itself:

```
class List<T> implements Iterable<T> {
    Node<T> head, tail;
    public Iterator<T> iterator() {
        return new ListIterator<T>(this);
    }
}
```

5.2 The iterator side

The **ListIterator** class holds a **current** node reference as its cursor. In its constructor, that cursor is initialized to the list's head. **hasNext()** simply checks whether the cursor has run past the end of the list (i.e. become null), and **next()** reads the current node's data before advancing the cursor to the following node:

```
class ListIterator<T> implements Iterator<T> {
    Node<T> current;
    ListIterator(List<T> list) { current = list.getHead(); }

    public boolean hasNext() { return current != null; }

    public T next() {
        T data = current.getData();
        current = current.getNext();
        return data;
    }

    public void remove() { throw new UnsupportedOperationException(); }
}
```

remove() is deliberately left unimplemented here (throwing **UnsupportedOperationException**), since this example only needs read-only traversal — **remove()** only needs a real implementation if removal during iteration is actually required.

5.3 Using it

Once these two pieces exist, the custom list behaves exactly like a built-in collection from the caller's point of view — it can be driven with a plain for-each loop, with no awareness that it isn't an **ArrayList** or **LinkedList** under the hood:

```
for (String s : myList) {
    System.out.println(s);
}
```

Given elements added in the order "abc", "mno", "pqr", "xyz", this loop prints each string on its own line, in that same insertion order — the list is walked head to tail exactly once, driven entirely by hasNext()/next() under the for-each syntax.

6. Summary

Piece	Responsibility
Data structure class	Implements Iterable<T>; its iterator() method returns a fresh Iterator instance
Iterator class (often inner)	Implements Iterator<T>; holds the cursor and implements hasNext(), next(), and (optionally) remove()
Cursor initialization	Depends on the structure — head node for a linked list, index 0 for an array-backed one
Access to the structure's internals	Direct, via qualified this, if inner class; via a constructor reference, if a standalone class
Result	The custom structure becomes usable in a for-each loop, indistinguishable from a built-in collection